Lab program-7:

7. Construct a C program to implement non-preemptive SJF algorithm

C lab_7.c > main()

```c
#include <stdio.h>

int main()
{
    int at[10], bt[10], pr[10];
    int n, i, j;
    int temp;
    int time = 0;
    int count;
    int over = 0;
    int sum_wait = 0;
    int sum_turnaround = 0;
    int start;

    float avgwait, avgturn;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    // Input Arrival Time and Burst Time
    for (i = 0; i < n; i++)
    {
        printf("\nEnter the Arrival Time and Burst Time for Process %d: ", i + 1);
        scanf("%d %d", &at[i], &bt[i]);
        pr[i] = i + 1;
    }

    // Sort by Arrival Time
    for (i = 0; i < n - 1; i++)
    {
        for (j = i + 1; j < n; j++)
        {
            if (at[i] > at[j])
            {
                temp = at[i];
                at[i] = at[j];
                at[j] = temp;

                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;

                temp = pr[i];
                pr[i] = pr[j];
                pr[j] = temp;
            }
        }
    }

    printf("\n--------------------------------------------------------------------------------\n");
    printf("Process\tArrival\tBurst\tStart\tEnd\tWaiting\tTurnaround\n");
    printf("--------------------------------------------------------------------------------\n");

    while (over < n)
    {
        count = 0;

        // Count processes that have arrived
        for (i = over; i < n; i++)
        {
            if (at[i] <= time)
                count++;
            else
                break;
        }

        // CPU Idle
        if (count == 0)
        {
            time = at[over];
            continue;
```

```
        {
            time = at[over];
            continue;
        }

        // Sort available processes by Burst Time
        if (count > 1)
        {
            for (i = over; i < over + count - 1; i++)
            {
                for (j = i + 1; j < over + count; j++)
                {
                    if (bt[i] > bt[j])
                    {
                        temp = at[i];
                        at[i] = at[j];
                        at[j] = temp;

                        temp = bt[i];
                        bt[i] = bt[j];
                        bt[j] = temp;

                        temp = pr[i];
                        pr[i] = pr[j];
                        pr[j] = temp;
                    }
                }
            }
        }

        start = time;
        time += bt[over];

        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
               pr[over],
               at[over],
               bt[over],
               start,
               time,
               time - at[over] - bt[over],
               time - at[over]);

        sum_wait += time - at[over] - bt[over];
        sum_turnaround += time - at[over];

        over++;
    }

    avgwait = (float)sum_wait / n;
    avgturn = (float)sum_turnaround / n;

    printf("-------------------------------------------------------------------------------\n");
    printf("\nAverage Waiting Time    = %.2f", avgwait);
    printf("\nAverage Turnaround Time = %.2f\n", avgturn);

    return 0;
}
```

OUTPUT:

```
PS C:\Users\SMILE\Documents\OS\Lab programs> ./lab_7.exe
Enter the number of processes: 3

Enter the Arrival Time and Burst Time for Process 1: 4 7

Enter the Arrival Time and Burst Time for Process 2: 2 6

Enter the Arrival Time and Burst Time for Process 3: 3 9

--------------------------------------------------------------------------------
Process Arrival Burst   Start   End     Waiting Turnaround
--------------------------------------------------------------------------------
P2      2       6       2       8       0       6
P1      4       7       8       15      4       11
P3      3       9       15      24      12      21
--------------------------------------------------------------------------------

Average Waiting Time    = 5.33
Average Turnaround Time = 12.67
```